

Universidad de O'Higgins
Escuela de Ingeniería — COM4602

Predictor de Juegos Steam

Informe de Análisis del Repositorio Git

Introducción a los Repositorios de Código Distribuido
Proyecto individual — Primer Semestre 2026

Estudiante: Eduardo Sáez Gattás

Fecha de entrega: 05/07/2026

Repositorio GitHub: github.com/HichiroXD/proyecto_git_1

2. Descripción del proyecto

El mercado de distribución digital en Steam es altamente competitivo, con miles de lanzamientos anuales. Para un desarrollador independiente resulta valioso entender, antes de invertir recursos en producción, qué características estructurales de un juego se asocian históricamente con una buena recepción de la comunidad. Este proyecto aborda ese problema mediante un modelo de clasificación multiclase que predice el nivel de éxito de un juego (Negative, Mixed o Positive) usando únicamente información disponible **antes del lanzamiento**: género, precio, plataformas soportadas, logros planeados y desarrollador, entre otras.

El objetivo principal es entrenar y comparar dos modelos de árboles de decisión (Decision Tree y Random Forest) sobre el historial de juegos ya lanzados en Steam, evaluando su capacidad de generalización mediante validación cruzada y métricas multiclase, y documentar honestamente tanto sus aciertos como sus limitaciones.

Funcionalidades principales

- Carga y merge automático de dos datasets de Kaggle (terencicp y fronkongames) mediante *AppID*, unificando ~57.000 juegos con columnas complementarias.
- Limpieza, codificación (One-Hot) y construcción de la variable objetivo multiclase con filtro de reseñas mínimas para reducir el sesgo temporal.
- Selección de variables por importancia acumulada (*CumulativeImportanceSelector*) encapsulada en un Pipeline de sklearn, evitando data leakage entre train y test.
- Entrenamiento y validación cruzada estratificada (5-fold) de Decision Tree y Random Forest con `class_weight='balanced'` para compensar el desbalance del catálogo de Steam.
- Evaluación final en test set (matrices de confusión, curvas ROC one-vs-rest, importancia de features) y selección del modelo con mejor desempeño.

El resultado final es orientativo: el modelo estima probabilidades históricas a partir de patrones del catálogo, no predice con certeza el éxito de un juego específico.

3. Estructura del repositorio

El repositorio se organiza en módulos con responsabilidad única, separando datos, código fuente, notebook de análisis e informe:

```
proyecto_git_1/
├── data/datasets/    # CSVs de Kaggle (excluidos del repo vía .gitignore)
├── src/
│   ├── config.py     # Constantes, rutas y umbrales centralizados
│   ├── data_loader.py # Carga y merge de los dos datasets
│   ├── feature_eng.py # Limpieza, encoding y target
│   ├── model.py      # Pipelines sklearn y entrenamiento
│   └── evaluation.py  # Métricas y visualizaciones multiclase
├── notebooks/analysis_final.ipynb # Análisis completo importando src/
├── reports/informe.pdf # Este informe
├── .gitignore
├── environment.yml
└── README.md
```

Esta separación entre `src/` (lógica reutilizable) y `notebooks/` (orquestración y narrativa del análisis) fue una decisión temprana del proyecto, visible desde los primeros commits (50e6c46, 9dba90d), y se mantuvo estable durante todo el desarrollo.

4. Evolución del desarrollo

El repositorio se inició como un repositorio local (git init) el 26 de junio de 2026 y se subió a GitHub ese mismo día, tras la creación de la estructura base de carpetas (data, notebooks, reports, src) y el archivo .gitignore (para excluir datasets, entornos virtuales y checkpoints de Jupyter).



```
MINGW64:/c/Users/saeze/OneDrive/Escritorio/Universidad/Introduccion a Repos/proyecto_git_1

saeze@Morrigan MINGW64 ~/OneDrive/Escritorio/Universidad/Introduccion a Repos
$ pwd
/c/Users/saeze/OneDrive/Escritorio/Universidad/Introduccion a Repos

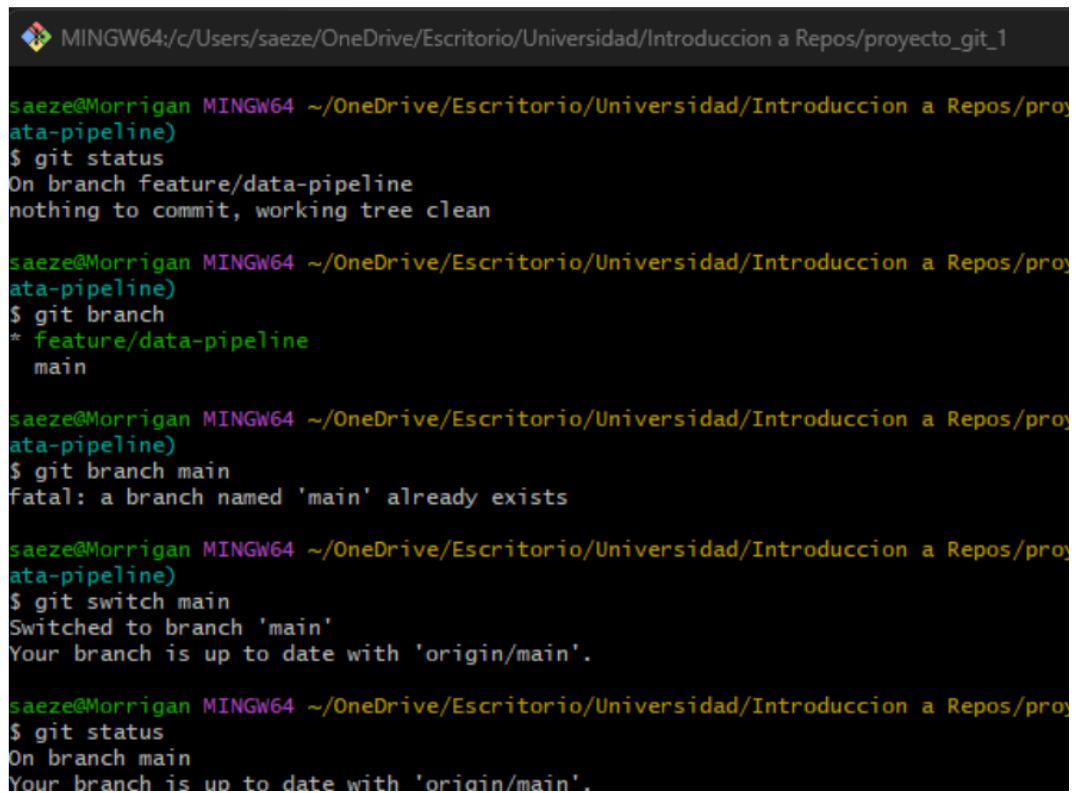
saeze@Morrigan MINGW64 ~/OneDrive/Escritorio/Universidad/Introduccion a Repos
$ mkdir proyecto_git_1

saeze@Morrigan MINGW64 ~/OneDrive/Escritorio/Universidad/Introduccion a Repos
$ ls
Historial_git.ipynb  Modulo_practico_branches_enunciados.ipynb  mi_proyecto_git/
Intro_git.ipynb     Recuperacion_gitignore.ipynb               proyecto_git_1/
```

Figura 1. Inicialización del repositorio local (git init) previo a la subida a GitHub.

La evolución cronológica del proyecto fue la siguiente:

- **26 de junio:** estructura del proyecto, .gitignore, entorno conda (environment.yml) y README inicial. 4 commits sobre main.
- **27–28 de junio:** apertura de la rama feature/data-pipeline para implementar la carga y el merge de los dos datasets de Kaggle; se integra a main mediante un Pull Request en GitHub (commit de merge cc1653f).
- **28–29 de junio:** continúa el desarrollo de feature/data-pipeline con la construcción del target multiclase, el filtro de reseñas mínimas y el encoding separado de ambos datasets.
- **2 de julio:** se abre hotfix/target-definition para ajustar de forma urgente los umbrales del target y el hiperparámetro RF_N_ESTIMATORS; en paralelo se desarrolla feature/modeling con los pipelines de Decision Tree y Random Forest, el selector de importancia acumulada y las funciones de evaluación (matriz de confusión, curvas ROC). En esta misma fecha ocurre el **conflicto de merge** entre main y hotfix/target-definition.
- **4 de julio:** se integran a main tanto feature/data-pipeline (acdd0aa) como feature/modeling (ac42c5b); se ejecuta el pipeline completo, se prueban 5, 4 y 3 clases para el target, y se fija la configuración final (3 clases + class_weight='balanced').
- **5 de julio:** se agrega el notebook de análisis completo (1b6162b) y se actualiza el README con resultados reales y conclusiones (5478845).



A terminal window with a dark background and light-colored text. The title bar at the top shows a Windows logo icon followed by the path: MINGW64:/c/Users/saeze/OneDrive/Escritorio/Universidad/Introduccion a Repos/proyecto_git_1. The terminal content shows a series of Git commands and their outputs. The user is in a directory named 'proyecto_git_1'. They run 'git status' and get the message 'On branch feature/data-pipeline' and 'nothing to commit, working tree clean'. Then they run 'git branch' and see a list with 'feature/data-pipeline' marked with an asterisk and 'main'. Next, they run 'git branch main' and receive a 'fatal: a branch named 'main' already exists' error. They then run 'git switch main' and are successfully switched to the 'main' branch, with a message 'Your branch is up to date with 'origin/main''. Finally, they run 'git status' again and get the same 'On branch main' and 'Your branch is up to date with 'origin/main'' message.

```
MINGW64:/c/Users/saeze/OneDrive/Escritorio/Universidad/Introduccion a Repos/proyecto_git_1

saeze@Morrigan MINGW64 ~/OneDrive/Escritorio/Universidad/Introduccion a Repos/proyecto_git_1
$ git status
On branch feature/data-pipeline
nothing to commit, working tree clean

saeze@Morrigan MINGW64 ~/OneDrive/Escritorio/Universidad/Introduccion a Repos/proyecto_git_1
$ git branch
* feature/data-pipeline
  main

saeze@Morrigan MINGW64 ~/OneDrive/Escritorio/Universidad/Introduccion a Repos/proyecto_git_1
$ git branch main
fatal: a branch named 'main' already exists

saeze@Morrigan MINGW64 ~/OneDrive/Escritorio/Universidad/Introduccion a Repos/proyecto_git_1
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

saeze@Morrigan MINGW64 ~/OneDrive/Escritorio/Universidad/Introduccion a Repos/proyecto_git_1
$ git status
On branch main
Your branch is up to date with 'origin/main'.
```

Figura 2. Creación y primer push de la rama feature/data-pipeline hacia GitHub.

5. Análisis del historial de Git

Métrica	Valor
Número total de commits	37
Días con actividad	7 (26, 27, 28 y 29 de junio; 2, 4 y 5 de julio)
Ramas utilizadas	main, feature/data-pipeline, feature/modeling, hotfix/target-definition
Merges realizados (commits de merge)	4 (cc1653f, acdd0aa, ac42c5b, 2095940) + 1 integración fast-forward
Three-way merge con conflicto manual	2095940 — conflicto en src/config.py
Archivos más modificados	src/config.py, src/model.py, src/evaluation.py, src/data_loader.py

Commits por día

Fecha	Commits	Hito principal
2026-06-26	4	Estructura base, .gitignore, entorno, README inicial
2026-06-27	1	Encode inicial de terencicp en data_loader
2026-06-28	4	Merge PR #1 (feature/data-pipeline → main)
2026-06-29	6	Target multiclase y filtro de reseñas mínimas
2026-07-02	9	Hotfix de umbrales + conflicto de merge + pipelines de modelado
2026-07-04	11	Merge de feature/data-pipeline y feature/modeling a main; ajuste a 3 clases
2026-07-05	2	Notebook final y README con resultados

Commits más relevantes

- **50e6c46** — Estructura del proyecto y módulos de trabajo (commit raíz).
- **cc1653f** — Merge pull request #1 from feature/data-pipeline (primer merge vía GitHub).
- **c8d826d** — Fix: ajuste de umbrales del target en hotfix/target-definition (origen del conflicto).
- **2095940** — Fix: resolución manual del conflicto de merge entre main y hotfix/target-definition.
- **acdd0aa / ac42c5b** — Integración final de feature/data-pipeline y feature/modeling a main.
- **c3371de / f32757f** — Reducción a 3 clases y adición de class_weight='balanced' (decisión basada en datos).
- **5478845** — Documentación final de resultados y conclusiones en el README.

Salida de git log --pretty=format

A continuación se muestra la salida solicitada del historial completo con fechas, autor y mensaje:

```

saeze@Morrigan MINGW64 ~/OneDrive/Escritorio/Universidad/Introduccion a Repos/proyecto_git_1 (main)
$ git log --pretty=format:"%h | %ad | %an | %s" --date=short
5478845 | 2026-07-05 | HichiroXD | docs: actualizar README con resultados reales y conclusiones
1b6162b | 2026-07-05 | HichiroXD | feat: agregar notebook de análisis completo
f32757f | 2026-07-05 | HichiroXD | feat: agregar class_weight='balanced' en ambos modelos
c3371de | 2026-07-05 | HichiroXD | feat: reducir a 3 clases y agregar class_weight balanced
921a2c1 | 2026-07-04 | HichiroXD | Update: Se actualizaron los código de config.py y feature_eng.py donde se actualizaron las clases y se reconsidero el sistema.
b465575 | 2026-07-04 | HichiroXD | fix: revertir umbrales MIN_REVIEWS_OVERWHELM y MIN_REVIEWS_VERY
ac42c5b | 2026-07-04 | HichiroXD | Merge branch 'feature/modeling'
acdd0aa | 2026-07-04 | HichiroXD | Merge branch 'feature/data-pipeline'
6604984 | 2026-07-04 | HichiroXD | Se implemento el código compare_model() para ver cual de los modelos funciona mejor para el análisis.
53bc2fd | 2026-07-04 | HichiroXD | Se implemento el código plot_feature_importance que genera graficos de los features mas importantes.
12da7b0 | 2026-07-04 | HichiroXD | Se implemento el código de plot_roc_curves() para estrategia One-vs-Rest.
43ae5b3 | 2026-07-04 | HichiroXD | Se implemento código de plot_confusion_matrix() para normalización de datos.
c1fe20f | 2026-07-04 | HichiroXD | Se implemento el código de print_report() y se agregaron mas librerias.
2095940 | 2026-07-02 | HichiroXD | Fix: Resolver conflicto de merge entre main y hotfix/target-definition.
f85329d | 2026-07-02 | HichiroXD | Config: Ajustar RF_N_ESTIMATORS = 200 a 300 y arreglo de directorios.
0a3d4e9 | 2026-07-02 | HichiroXD | config: ajustar RF_N_ESTIMATORS a 150 en main y cambios en directorios.
89616cf | 2026-07-02 | HichiroXD | Fixet: Arreglos en directorio para llamada de datos desde config hasta modelo.
1c0306a | 2026-07-02 | HichiroXD | Feat: Se implemento el código de cross_validate_pipeline() y __main__ para ejecutar el cross validate y un código para ejecutarlo.
1d56ea2 | 2026-07-02 | HichiroXD | Feat: Se implemento código de split_data() para conjunto de train y test.
f10e39b | 2026-07-02 | HichiroXD | Feat: Se implemento el código de build_pipeline_dt() y build_pipeline_rf().
f9a27bb | 2026-07-02 | HichiroXD | Feat: Implementar código Cumulative Importance Selector en modelo.
c8d826d | 2026-07-02 | HichiroXD | Fix: Se ajustaron los umbrales del target multiclase para dejarlo mas exigente e inclusivo.
3fb514f | 2026-06-29 | HichiroXD | Implementar prepare_datasets con el pipeline completo
e673e9e | 2026-06-29 | HichiroXD | Implementar el otro encode de fronkongames
a7af056 | 2026-06-29 | HichiroXD | Implementar cambio en encode para los datasets por separado
d1a13e5 | 2026-06-29 | HichiroXD | Implementar extractor de features en base al tiempo
37a63f9 | 2026-06-29 | HichiroXD | Implementar la variable objetivo built_target
09b69fa | 2026-06-29 | HichiroXD | Implementar filtro de reseñas minimo para calidad del target
b1bf144 | 2026-06-28 | HichiroXD | Implementacion dentro del data_loader (merge_datasets, load_add y get_merge_stats)
ca0cb70 | 2026-06-28 | HichiroXD | Implementar dentro de data_loader los _pivot para las tablas de terencicp
cc1653f | 2026-06-28 | Eduardo Sáez Gattás | Merge pull request #1 from HichiroXD/feature/data-pipeline
5afb31e | 2026-06-28 | HichiroXD | Agregar Fronkon_cols_merge y load_fronkongames
ce2e5ae | 2026-06-27 | HichiroXD | Implementar load_terencicp() para proceso dentro de la rama Feature
f2ff39a | 2026-06-26 | HichiroXD | Cambios y arreglos al README
7453f0f | 2026-06-26 | HichiroXD | README inicial con descripción del problema y estructura del Repo
9dba90d | 2026-06-26 | HichiroXD | Creación de .gitignore, subida de datasets y modulos
50e6c46 | 2026-06-26 | E.S.G | Estructura del proyecto y modulos de trabajo

saeze@Morrigan MINGW64 ~/OneDrive/Escritorio/Universidad/Introduccion a Repos/proyecto_git_1 (main)
$

```

Figura 3. Salida de git log --pretty=format:"%h | %ad | %an | %s" --date=short.

Visualización completa del historial (git log --graph)

6. Análisis de integración de cambios

Ramas utilizadas y objetivo de cada una

Rama	Objetivo
main	Rama principal e íntegra del proyecto; recibe los cambios ya integrados y probados.
feature/data-pipeline	Desarrollo aislado de la carga y merge de los dos datasets de Kaggle (data_loader.py, feature_eng.py): pivoteo de categorías/tags, filtro de reseñas mínimas y construcción del target.
feature/modeling	Desarrollo aislado de los pipelines de modelado (model.py) y evaluación (evaluation.py): Decision Tree, Random Forest, selección de features, validación cruzada, matrices de confusión y curvas ROC.
hotfix/target-definition	Rama de corrección rápida para ajustar los umbrales del target multiclase y el hiperparámetro RF_N_ESTIMATORS directamente sobre config.py, en paralelo al desarrollo de las otras dos ramas.

El conflicto de merge y su resolución manual

El conflicto surgió porque dos ramas modificaron de forma independiente el mismo parámetro en src/config.py a partir de un mismo ancestro común (commit c8d826d, resultado de una integración previa por fast-forward de hotfix/target-definition en main): sobre main se hizo el commit 0a3d4e9, fijando RF_N_ESTIMATORS = 150; en paralelo, sobre hotfix/target-definition se hizo el commit f85329d, fijando el mismo parámetro en 300 (ajustado desde 200). Al ejecutar git merge hotfix/target-definition desde main, Git no pudo reconciliar automáticamente ambos valores sobre la misma línea del archivo y reportó CONFLICT (content): Merge conflict in src/config.py.


```
MINGW64:/c/Users/saeze/OneDrive/Escritorio/Universidad/Introduccion a Repos/proyecto_git_1

saeze@Morrigan MINGW64 ~/OneDrive/Escritorio/Universidad/Introduccion a Repos/proy
$ git merge hotfix/target-definition
Auto-merging src/config.py
CONFLICT (content): Merge conflict in src/config.py
Automatic merge failed; fix conflicts and then commit the result.

saeze@Morrigan MINGW64 ~/OneDrive/Escritorio/Universidad/Introduccion a Repos/proy
NG)
$
```

```
MINGW64:/c/Users/saeze/OneDrive/Escritorio/Universidad/Introduccion a Repos/proyecto_git_1

saeze@Morrigan MINGW64 ~/OneDrive/Escritorio/Universidad/Introduccion a Repos/proy
$ git merge hotfix/target-definition
Auto-merging src/config.py
CONFLICT (content): Merge conflict in src/config.py
Automatic merge failed; fix conflicts and then commit the result.

saeze@Morrigan MINGW64 ~/OneDrive/Escritorio/Universidad/Introduccion a Repos/proy
NG)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)
```

Figura 5. Conflicto de merge en src/config.py y su resolución manual (commit 2095940).

La resolución se realizó manualmente editando src/config.py, eligiendo un valor final para RF_N_ESTIMATORS que conciliara ambos criterios (mayor estabilidad del Random Forest sin encarecer demasiado el entrenamiento), eliminando las marcas de conflicto (<<<<<<, =====, >>>>>>) y confirmando la resolución con git add src/config.py seguido de git commit -m "Fix: Resolver conflicto de merge entre main y hotfix/target-definition.", generando el commit 2095940. Este commit tiene dos padres (main y hotfix/target-definition) con un ancestro común distinto de ambos, cumpliendo la definición de **three-way merge**.

Un segundo merge relevante: feature/modeling → main

La integración de feature/modeling en main (commit ac42c5b) se resolvió automáticamente mediante la estrategia recursiva *ort* de Git, sin conflictos, ya que las modificaciones de ambas ramas afectaban archivos y secciones distintas (evaluation.py y model.py):

```

MINGW64/c/Users/saeze/OneDrive/Escritorio/Universidad/Introduccion a Repos/proyecto_git_1
saeze@Morrigan MINGW64 ~/OneDrive/Escritorio/Universidad/Introduccion a Repos/proyecto_g
$ git merge feature/modeling
Merge made by the 'ort' strategy.
 src/evaluation.py | 185 ++++++-----
 src/model.py      | 185 ++++++-----
 2 files changed, 343 insertions(+), 27 deletions(-)

saeze@Morrigan MINGW64 ~/OneDrive/Escritorio/Universidad/Introduccion a Repos/proyecto_g
$ git status
On branch main
Your branch is ahead of 'origin/main' by 11 commits.
  (use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   src/config.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        run_diagnostico.py

no changes added to commit (use "git add" and/or "git commit -a")

saeze@Morrigan MINGW64 ~/OneDrive/Escritorio/Universidad/Introduccion a Repos/proyecto_g
$ git status
On branch main
Your branch is ahead of 'origin/main' by 11 commits.
  (use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   src/config.py

no changes added to commit (use "git add" and/or "git commit -a")

saeze@Morrigan MINGW64 ~/OneDrive/Escritorio/Universidad/Introduccion a Repos/proyecto_g
$ git add src/config.py

```

Figura 6. Merge limpio de feature/modeling en main mediante la estrategia 'ort'.

7. Visualización del repositorio

Además del gráfico de git log --graph (Figura 4, sección 5), GitHub expone en su pestaña *Insights* tres visualizaciones que resumen la evolución del repositorio: la actividad de commits en el tiempo, la frecuencia de líneas agregadas/eliminadas por semana, y el grafo de red de ramas. Las tres se muestran a continuación, obtenidas directamente desde github.com/HichiroXD/proyecto_git_1.

Insights → Commits over time

La vista semanal (4 abr – 4 jul 2026) muestra con claridad las dos ráfagas de trabajo ya identificadas en la sección 5: una semana con actividad moderada (semana del 22 de junio), una semana con un pico muy marcado de más de 20 commits (semana del 29 de junio, que concentra el desarrollo de feature/data-pipeline, el hotfix y buena parte de feature/modeling), y luego una caída a un nivel bajo y estable (semana del 6 de julio, cierre del proyecto). El panel de contribuidor confirma que HichiroXD concentra 33 commits reconocidos por GitHub, con 2.441 líneas agregadas y 180 eliminadas.



Figura 7. GitHub Insights — Commits over time y resumen de contribuciones de HichiroXD.

Insights → Code frequency

El gráfico de frecuencia de código confirma el mismo patrón desde otro ángulo: las adiciones crecen semana a semana (de ~570 a ~830 y finalmente a ~980 líneas), mientras que las eliminaciones más importantes ocurren en la semana del 29 de junio (~320 líneas), coincidiendo con la reestructuración del target multiclase y el refactor del encoding de fronkongames documentados en los commits a7af056 y e673e9e. La última semana (6 de julio) casi no presenta eliminaciones, lo que es consistente con que ese período correspondió mayormente a agregar funcionalidad nueva (pipelines de modelado, notebook final) en vez de reescribir código existente.

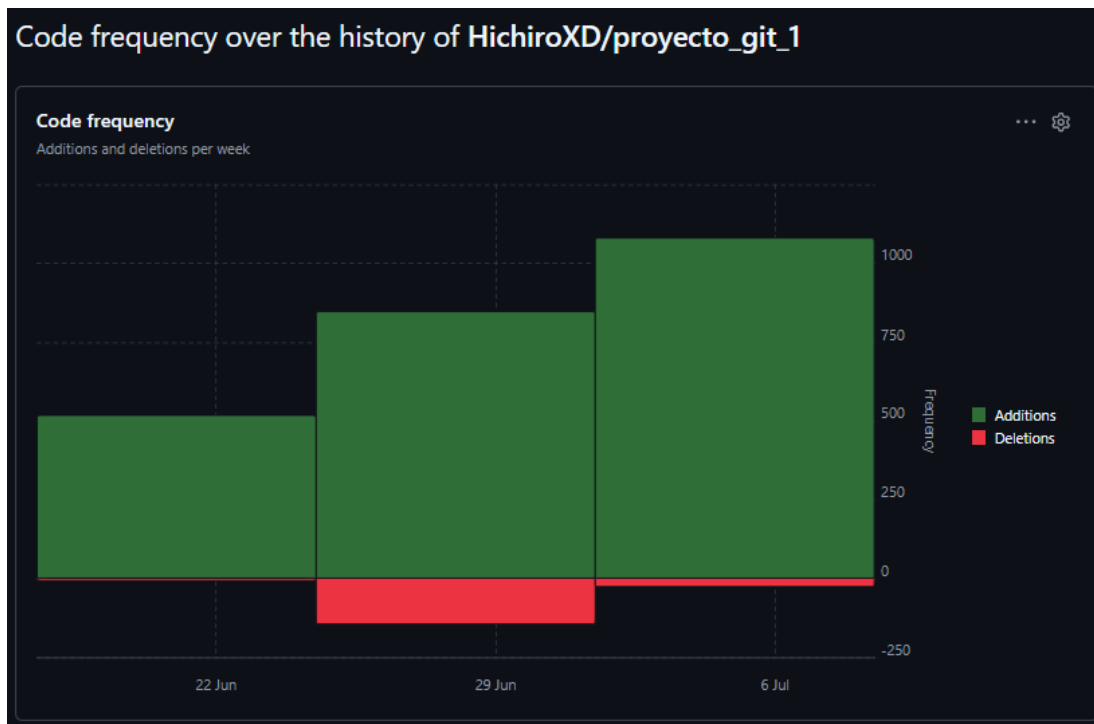


Figura 8. GitHub Insights — Code frequency (adiciones y eliminaciones por semana).

Insights → Network graph

El grafo de red muestra, en la ventana de los días más recientes del proyecto (4 y 5 de julio), cómo la rama `feature/modeling` (verde) y otra rama de desarrollo (morada) convergen sobre `main` (blanco/azul) mediante los commits de merge, quedando finalmente una única línea de historia activa en `main`. Esta vista es la versión gráfica e interactiva de GitHub para lo mismo que se documentó por consola en la Figura 4.

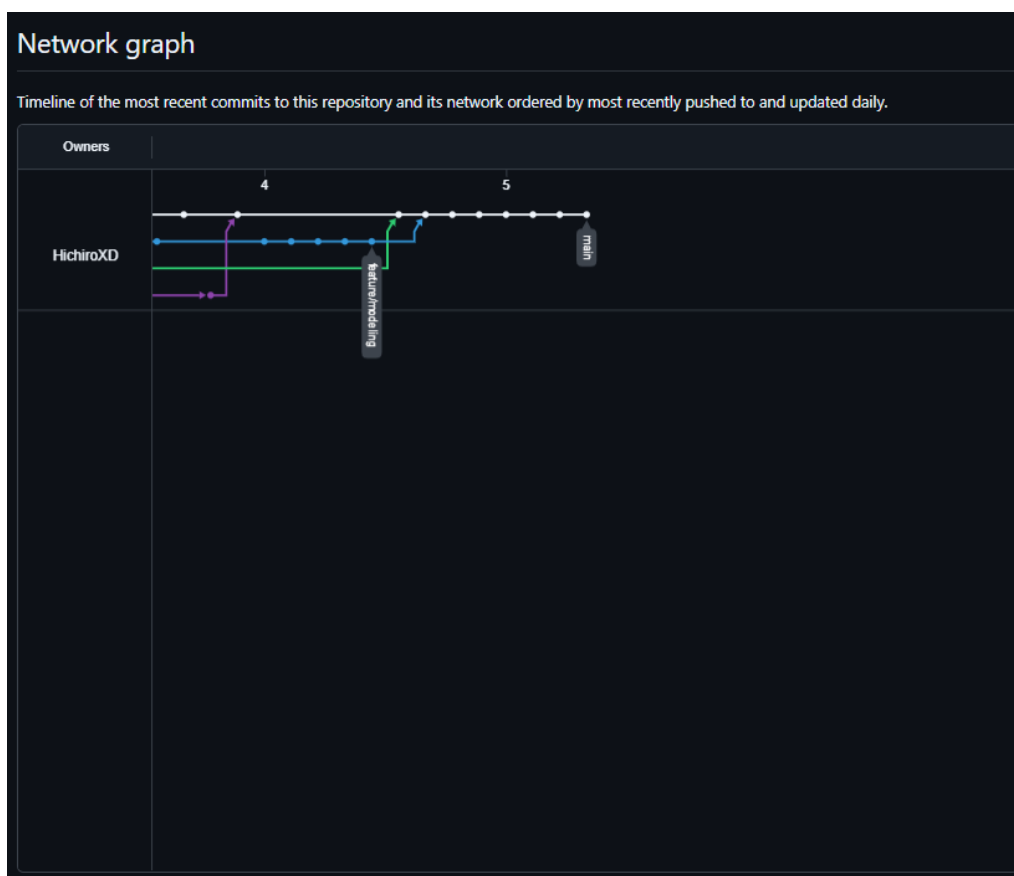


Figura 9. GitHub Insights — Network graph mostrando la convergencia de ramas hacia main.

8. Análisis de métricas

Commits por día

La actividad no fue uniforme: se concentró en dos ráfagas de trabajo (29 de junio con 6 commits y sobre todo 2 y 4 de julio, con 9 y 11 commits respectivamente) separadas por un día sin actividad registrada (30 de junio y 1 de julio). Esto refleja un patrón típico de desarrollo por sprints cortos: una sesión dedicada a la construcción del pipeline de datos, otra al hotfix urgente de configuración, y una sesión intensa final dedicada a integrar ambas ramas de features y ajustar el modelo. El promedio es de ~5,3 commits por día activo, lo que indica commits razonablemente atómicos en vez de commits gigantes concentrados al final del período, cumpliendo con la exigencia de avances reales y progresivos.

Volumen de cambios (líneas agregadas/eliminadas)

Según GitHub Insights, el repositorio registra 2.441 líneas agregadas y 180 eliminadas bajo el contribuidor HichiroXD, con 33 commits reconocidos en el panel de contribuciones — un número algo menor a los 37 commits que arroja `git log --all` localmente. La diferencia se explica principalmente porque el commit de merge del Pull Request #1 (cc1653f) quedó autoría como Eduardo Sáez Gattás (nombre de perfil de GitHub) en vez de HichiroXD, y porque la ventana de *Insights* → *Commits over time* está fijada al rango 4 abr–4 jul 2026, lo que deja fuera del gráfico semanal los dos últimos commits del 5 de julio aunque sí quedan contabilizados en el total de líneas. Esta clase de pequeños desajustes entre las estadísticas cacheadas de GitHub y la fuente de verdad (`git log local`) es normal y no afecta la validez del historial: la proporción entre adiciones y eliminaciones (más de 13:1) confirma que el proyecto creció principalmente agregando funcionalidad nueva,

con relativamente poco código descartado o reescrito.

Ramas y merges

El uso de tres ramas de desarrollo (dos *feature* y un *hotfix*) por sobre el mínimo exigido de dos muestra una separación de responsabilidades clara: los datos y el modelo se desarrollaron en paralelo sin bloquearse mutuamente, mientras que el *hotfix* permitió corregir un parámetro urgente sin detener el trabajo en curso en las otras ramas. El precio de este paralelismo fue, precisamente, el conflicto documentado en la sección 6: dos ramas modificando el mismo parámetro de configuración desde un ancestro común. Esto ilustra un caso real (no artificial) de por qué son necesarias las prácticas de sincronización frecuente y de comunicación entre ramas activas.

Hotspots del repositorio

El archivo con más historial de cambios y el único que fue tocado por las tres ramas de desarrollo es `src/config.py` — es, además, el único archivo que sufrió un conflicto de merge, lo cual es coherente: al centralizar constantes y umbrales (según el propio README, «rutas y umbrales centralizados»), cualquier ajuste de hiperparámetros o de reglas del target termina modificando este archivo, aumentando la probabilidad de choques entre ramas. `src/model.py` y `src/evaluation.py` también concentran muchos cambios, pero al ser modificados predominantemente por una sola rama (*feature/modeling*) no generaron conflictos al integrarse.

Autoría de los commits

Se observan tres variantes del nombre de autor en el historial: E.S.G en el commit raíz, HichiroXD (usuario de GitHub configurado como `user.name` local) en la gran mayoría de los commits, y Eduardo Sáez Gattás en el commit de merge del Pull Request #1 (cc1653f), generado directamente desde la interfaz web de GitHub, que usa el nombre de perfil en lugar de la configuración local de Git. Esto es consistente con un único autor trabajando desde distintas herramientas (terminal local vía Git Bash/MINGW64 y la interfaz web de GitHub para aceptar el Pull Request), y no representa una inconsistencia de autoría real.

9. Reflexión final

Desarrollar este proyecto usando ramas, Pull Requests y un *hotfix* paralelo permitió experimentar de primera mano varias ideas que en clases se ven de forma más abstracta. La primera es que las ramas no son solo una forma de "organizar" el trabajo, sino una herramienta real para aislar riesgo: mientras *feature/modeling* avanzaba en los pipelines de Decision Tree y Random Forest, *feature/data-pipeline* podía seguir refinando el target multiclase sin que un error en una rama rompiera el trabajo de la otra.

La segunda es que los conflictos de merge no son un evento excepcional reservado para equipos grandes: bastó con tener dos ramas (*main* y *hotfix/target-definition*) tocando el mismo parámetro de configuración a partir de un ancestro común para generar un conflicto real en `src/config.py`. Resolverlo manualmente obligó a entender exactamente qué representaba cada valor en conflicto (`RF_N_ESTIMATORS = 150` vs. `300`) antes de decidir cuál conservar, en vez de simplemente aceptar un lado a ciegas.

También quedó claro el valor de `git log --graph` y de herramientas como los Insights de GitHub para diagnosticar la salud de un repositorio: permiten ver de un vistazo qué ramas siguen vivas, dónde ocurrieron los merges y qué archivos concentran más cambios (hotspots), información que habría sido difícil de reconstruir solo leyendo mensajes de commit sueltos.

Finalmente, mantener commits frecuentes y descriptivos —incluso en una rama de hotfix pensada para ser rápida— resultó clave para poder reconstruir, semanas después, por qué se tomó cada decisión (por ejemplo, la reducción de 5 a 3 clases documentada en los commits c3371de y f32757f, o el ajuste de umbrales en b465575). El historial de Git terminó funcionando, en la práctica, como una bitácora técnica del proyecto tan útil como el propio README.

Declaración de uso de IA: este informe fue redactado con asistencia de Claude (Anthropic) a partir de la evidencia real del repositorio (capturas de consola, salidas de git log y el README/notebook del proyecto) proporcionada por el estudiante. El análisis, las decisiones de desarrollo, las ramas, los merges y la resolución del conflicto fueron realizados y ejecutados por el estudiante.